PART I: DATABASE FOUNDATIONS • Chapter 3

Data Models: Evolution, Comparison, and Modeling Levels

CHAPTER 3 ROADMAP & LEARNING OBJECTIVES

Why This Matters: *Why did E.F. Codd win the Turing Award for replacing pointer-based Hierarchical and Network databases with the Relational Model? And why do modern systems combine PostgreSQL (Relational/ORDBMS) with MongoDB (Document), Redis (Key-Value), Cassandra (Wide-Column), and Neo4j (Graph)? Mastering data models lets you design the right schema for any workload.*

Prerequisites: Chapters 1 & 2 (DBMS Fundamentals, Three-Schema Architecture, and Data Independence).

Learning Objectives (What You Will Master in This Chapter):

- Define a Data Model formally in terms of its three essential components: Structural part, Manipulative part, and Integrity Constraints.
- Classify data models across the three abstraction categories: Conceptual (High-Level), Representational/Logical, and Physical (Low-Level).
- Trace the historical evolution from Hierarchical (IBM IMS) and Network (CODASYL) models to the Relational Model, OODBMS/ORDBMS, and modern NoSQL models.
- Represent a Many-to-Many (M:N) Student–Course relationship side-by-side across Hierarchical, Network, Relational, and Document models.
- Progress systematically from a Conceptual ER design to a Logical Relational schema and finally a Physical SQL deployment.
- Select the optimal data model in system design interviews and write hybrid Object-Relational PostgreSQL DDL using Composite Types, Arrays, and JSONB.

Contents

3 Data Models: Evolution, Comparison, and Modeling Levels **1**

3.1 What Is a Data Model and What Does It Contain? **3**

3.1.1 The Three Essential Components of Every Data Model **3**

3.1.2 Three Categories of Data Models (Conceptual, Logical, Physical) **3**

3.2 Evolution of Data Models: From Legacy Pointers to Modern Engines **4**

3.2.1 Hierarchical Model (1960s — IBM IMS Tree Structure) **4**

3.2.2 Network Model (1969 — CODASYL DBTG Directed Graph) **5**

3.2.3 Relational Model (1970 — E.F. Codd's Mathematical Relations) **6**

3.2.4 Object-Oriented (OODBMS) and Object-Relational (ORDBMS) Models **6**

3.2.5 Modern Semi-Structured & NoSQL Models (Document, KV, Wide-Column, Graph) **7**

3.3 Dedicated Comparison Spread: The Seven Major Data Models **8**

3.3.1 Worked Example 3.1: Representing an M:N Relationship Across 4 Models **8**

3.3.2 Table 3.1: Master Comparison of the 7 Major Data Models **9**

3.4 Conceptual vs. Logical vs. Physical Data Modeling **10**

3.4.1 Table 3.2: Side-by-Side Comparison of the Three Modeling Stages **11**

3.4.2 Worked Example 3.2: End-to-End Walkthrough (Conceptual -> Logical -> Physical) **11**

3.5 Choosing the Right Data Model in System Design Interviews **12**

3.5.1 Worked Example 3.3: Hybrid Object-Relational Modeling in PostgreSQL **13**

3.5.2 Worked Example 3.4: Polyglot Persistence for a Campus Super-App **14**

3.6 Chapter 3 Summary & One-Page Rapid-Revision Cheat Sheet **15**

3.7 Chapter 3 Exam, GATE & Placement Question Bank (Parts A–H) **17**

3.8 Chapter 3 Complete Answer Key & Worked Solutions **21**

3.1 What Is a Data Model and What Does It Contain?

A Data Model is an integrated collection of mathematical and conceptual tools used to describe the structure of a database, the operations allowed on that data, and the constraints that keep the data valid.

3.1.1 The Three Essential Components of Every Data Model

1. Structural Part (Data Structures): Defines how data is conceptually organized and perceived by users (e.g., 2D tables/relations in the Relational Model, parent–child trees in the Hierarchical Model, JSON documents in the Document Model, or nodes and edges in a Graph Model).
2. Manipulative Part (Operations): Defines the set of formal operators used to retrieve and update data (e.g., Relational Algebra operators σ, π, ⋈ in the Relational Model; tree-traversal GET NEXT WITHIN PARENT in IMS; or Cypher pattern matching in Neo4j).
3. Integrity Constraints Part (Validation Rules): Defines structural rules that every valid database instance must obey (e.g., Primary Key uniqueness and Foreign Key referential integrity in the Relational Model, or strict single-parent tree rules in the Hierarchical Model).

3.1.2 Three Categories of Data Models (Aligned with ANSI/SPARC)

- 1. High-Level / Conceptual Data Models (Entity-Relationship / EER): Close to how business stakeholders think; models Entities, Attributes, and Relationships without tying to any specific DBMS.
- 2. Representational / Implementation (Logical) Data Models (Relational, Network, Hierarchical, Document): Understandable by developers while directly implementable in a commercial DBMS.
- 3. Low-Level / Physical Data Models: Describe byte-level record formats, slotted page headers, B+ Tree node layouts, and disk block access paths.

3.2 Evolution of Data Models: From Legacy Pointers to Modern Engines

3.2.1 The Hierarchical Data Model (1960s — IBM IMS)

Developed by IBM and North American Rockwell for the Apollo Moon program, the Hierarchical Model organizes records as an upside-down tree of parent and child segments:

- Strict 1:N Parent–Child Rule: Every child record has at most ONE parent record (except the root, which has none), while a parent can have multiple children. Natural for strict 1:N hierarchies (Department -> Course -> Section).
- Fatal Flaw on M:N Relationships: Because a child cannot have two parents in a strict tree, modeling a Many-to-Many relationship (e.g., Student enrolls in many Courses, and a Course has many Students) requires either duplicating Student records under every Course (causing massive redundancy and update anomalies) or maintaining complex "virtual pointer segments".
- Zero Physical Data Independence: Queries require procedural tree navigation (GET UNIQUE, GET NEXT WITHIN PARENT). If the tree hierarchy is reorganized, every COBOL/C application program breaks!

3.2.2 The Network Data Model (1969 — CODASYL DBTG)

Standardized by the Conference on Data Systems Languages (CODASYL), the Network Model generalized the strict tree into a directed graph using Owner–Member Sets (1:N links):

- Multiple Parents Allowed: A record type (e.g., Enrollment intersection record) can simultaneously be a member of two different Owner sets (owned by Student AND owned by Course), solving M:N relationships without duplicating Student data.
- Fatal Flaw — Circular Pointer Chains ("Spaghetti Navigation"): Records are connected on disk via physical circular linked lists of disk pointers (Next, Prior, and Owner pointers). Programmers had to manually navigate pointer chains using currency indicators (FIND FIRST, FIND NEXT WITHIN SET). A single corrupted disk pointer broke the entire ring!

1. HIERARCHICAL (IBM IMS)	Course(CS301) ---> Dup Student(Asha), Student(Ravi) | Course(CS201) ---> Dup Student(Asha) [Redundant Tree!]
2. NETWORK (CODASYL)	Student(Asha) --[Owner Set Pointer Ring]--> Enrollment(Grade A) <--[Owner Set Pointer Ring]-- Course(CS301)
3. RELATIONAL (E.F. Codd)	Student(StudentID=1) <--- Logical Value Match (FK = PK) ---> Enrollment(StudentID=1, CourseID="CS301", Grade="A")
4. DOCUMENT (MongoDB)	{ "_id": 1, "name": "Asha", "enrollments": [{ "courseId": "CS301", "grade": "A" }, { "courseId": "CS201", "grade": "A+" }] }

Figure 3.1 & 3.2: Four-Panel Evolution (Hierarchical Tree vs. Network Graph vs. Relational Tables vs. Document JSON) & CODASYL Pointer Ring vs. Logical Foreign Keys.

3.2.3 The Relational Data Model (1970 — E.F. Codd's Revolution)

In his landmark 1970 paper "A Relational Model of Data for Large Shared Data Banks," Edgar F. Codd replaced physical disk pointers with mathematical Relations (2D tables) and value-based Foreign Keys:

- Value-Based Relationships (No User-Visible Pointers!): Two tables are linked purely by matching attribute values (e.g., Student.DeptID = Department.DeptID), not by hardcoded disk addresses.
- Declarative Queries & Set-At-A-Time Processing: Instead of record-at-a-time pointer loops, users write declarative SQL based on Relational Algebra and Relational Calculus, achieving full Physical Data Independence.

3.2.4 Object-Oriented (OODBMS) and Object-Relational (ORDBMS) Models

In the 1980s–1990s, CAD/CAM and multimedia systems needed complex nested objects and inheritance. Pure OODBMS (ObjectStore, db4o) stored programming objects directly with Object IDs (OIDs) but lacked declarative SQL tooling. Object-Relational DBMS (ORDBMS — PostgreSQL, Oracle, Informix) won by extending the Relational Model with User-Defined Types (UDTs), Arrays, JSONB, table inheritance, and spatial indexes (PostGIS) while preserving full SQL and ACID.

3.2.5 Modern Semi-Structured and NoSQL Data Models (2000s–Present)

Web-scale applications with massive horizontal write throughput, flexible schemas, or deep graph traversals led to four specialized families of NoSQL ("Not Only SQL") data models:

1. Document Data Model (MongoDB, Couchbase, Firestore): Stores self-describing hierarchical documents in JSON/BSON format. Ideal for product catalogs, CMS articles, and user profiles where related nested arrays can be fetched in a single read without multi-table joins.
2. Key–Value Data Model (Redis, Amazon DynamoDB, Memcached): An ultra-fast hash map associating a unique Key with an opaque Value (string, JSON, hash, or sorted set). Delivers sub-millisecond O(1) lookups for session stores, caching, rate limiters, and leaderboards.
3. Wide-Column / Column-Family Data Model (Apache Cassandra, Google Bigtable, HBase): Organizes data by Row Key into column families where rows are sorted lexicographically and stored in LSM-Trees (Log-Structured Merge-Trees). Built for massive write-heavy time-series, IoT telemetry, and chat message history.
4. Graph Data Model (Neo4j, Amazon Neptune, JanusGraph): Models entities as Nodes (Vertices) and relationships as first-class directed Edges with properties (Property Graph). Uses Index-Free Adjacency so multi-hop traversals (e.g., 5-hop fraud detection or social "friends of friends") run in O(k) local hops rather than expensive 5-way SQL self-joins.

> **CONCEPT | Why Graph Databases Beat Relational Joins on Deep Multi-Hop Queries**
>
> In a relational table of 100M users, finding a 5-hop connection requires 5 self-joins of the Friendship table (O(log N) B+ Tree lookups per hop). In a Property Graph with Index-Free Adjacency, each node holds direct memory references to its adjacent edges, traversing only the local neighborhood!

3.3 Dedicated Comparison Spread: The Seven Major Data Models

3.3.1 Worked Example 3.1: Representing an M:N Relationship Across 4 Models

Suppose Asha (StudentID 1) and Ravi (StudentID 2) both enroll in CS301 (Database Systems), and Asha also enrolls in CS201 (Data Structures). Compare how four data models represent this M:N relationship:

1. Hierarchical Model: Because a child segment can have only one parent, placing Student under Course forces Asha's entire record (Name, CGPA, City) to be duplicated under both CS301 and CS201 (or requires proprietary IBM IMS logical twin pointers).
2. Network Model: Creates three record types (Student, Course, and an intersection record Enrollment) linked by two CODASYL Owner–Member pointer sets (Student_Enrolls and Course_Roster). Avoids duplication, but queries must traverse circular pointer chains.
3. Relational Model: Creates three normalized relations—Student(StudentID PK, Name), Course(CourseID PK, Title), and junction table Enrollment(StudentID FK, CourseID FK, Grade)—linked cleanly by matching primary/foreign key values.
4. Document Model (MongoDB): Embeds an array of course enrollments inside Asha's student document for 1-read student transcripts, or references course ObjectIds when courses have thousands of students.

Neo4j Cypher Query vs. SQL for a 2-Hop "Co-Enrolled Classmates" Query

```
-- Neo4j Property Graph (Cypher): Find classmates enrolled in the same course as Asha
MATCH (s:Student {name: 'Asha'})-[:ENROLLED_IN]->(c:Course)<-[:ENROLLED_IN]-(peer:Student)
WHERE peer.studentId <> s.studentId
RETURN DISTINCT peer.name, c.title;

-- Equivalent Relational SQL (Requires 2 Joins on Enrollment):
SELECT DISTINCT s2.Name, c.Title
FROM Student s1
JOIN Enrollment e1 ON s1.StudentID = e1.StudentID
JOIN Enrollment e2 ON e1.SectionID = e2.SectionID
JOIN Student s2    ON e2.StudentID = s2.StudentID
JOIN Course c      ON c.CourseID = 'CS301'
WHERE s1.Name = 'Asha' AND s2.StudentID <> s1.StudentID;
```

Table 3.1: Master Comparison of the Seven Major Data Models

Data Model	Core Structure	Relationships	Schema Flexibility	Representative DBMS & Best Use Case
Hierarchical (1960s)	Tree (Parent–Child Segments)	Physical Parent–Child Pointers (1:N only)	Rigid (Predefined tree)	IBM IMS, Windows Registry — Legacy mainframe banking/aviation
Network (1969)	Directed Graph (Owner–Member Sets)	Circular Linked-List Pointer Chains (1:N, M:N)	Rigid (CODASYL DDL)	IDMS, TurboIMAGE — High-speed legacy telecom/manufacturing
Relational (1970)	2D Relations (Tables, Tuples, Attributes)	Value-Based Foreign Keys (PK = FK)	Strict Schema-on-Write	PostgreSQL, MySQL, Oracle, SQL Server — ERP, Banking, OLTP
Object-Relational (1990s)	Relations + UDTs, Arrays, JSONB, Spatial	Foreign Keys + Nested Composite Types	Hybrid (Strict + JSONB)	PostgreSQL, Oracle — Geospatial (PostGIS), Hybrid SaaS
Document (NoSQL)	Nested JSON / BSON Documents	Embedded Sub-Documents or ID References	Flexible Schema-on-Read	MongoDB, Firestore, Couchbase — Catalogs, CMS, User Profiles
Key–Value (NoSQL)	Hash Map (Key -> Value / Blob / Struct)	None (Application-level key lookups)	Schema-less Values	Redis, DynamoDB — Caching, Session Store, Rate Limiting
Wide-Column & Graph	Column Families (LSM) / Nodes & Edges	Row Key Clustering / Index-Free Edges	Dynamic Columns / Properties	Cassandra (IoT/Chat logs), Neo4j (Fraud rings, Social Graphs)

EXAM CHECKPOINT | Key Difference Between Hierarchical, Network, and Relational Models

- Hierarchical = Tree (1 parent per child, physical pointers, procedural navigation).
- Network = Graph (multiple owner sets per member, circular pointer chains, procedural navigation).
- Relational = Tables (any relationship via value-based Foreign Keys, declarative SQL, full physical data independence).

3.4 Conceptual vs. Logical vs. Physical Data Modeling

When designing a new database from scratch, professional database engineers never jump straight to writing CREATE TABLE statements. Instead, they refine the design across three progressive modeling stages:

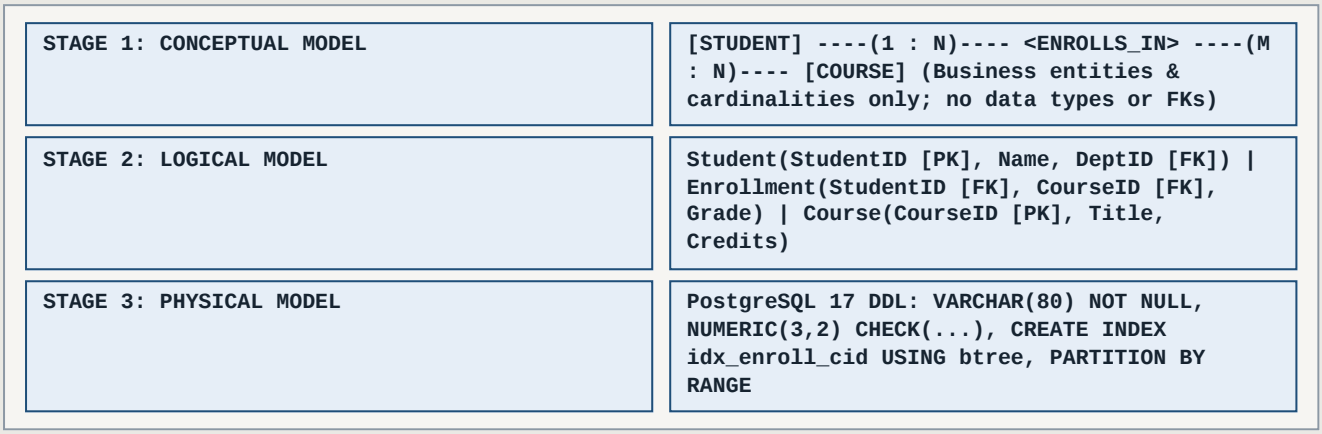


Figure 3.3: Progressive Refinement of UniVerseDB Across Conceptual, Logical, and Physical Data Modeling.

Table 3.2: Detailed Comparison of Conceptual, Logical, and Physical Data Models

Design Aspect	Conceptual Data Model	Logical Data Model	Physical Data Model
Primary Goal	Capture business entities & rules with stakeholders	Define normalized table structures & keys cleanly	Optimize SQL DDL, types, indexes & partitions for a target DBMS
DBMS Dependency	100% DBMS-Independent	DBMS-Family Specific (e.g., Relational)	DBMS-Engine & Version Specific (e.g., PostgreSQL 17)
Entities & Tables	High-level Entity Boxes	Normalized Relations (3NF / BCNF)	Physical Tables, Partitioned Tables, Materialized Views
M:N Relationships	Shown directly as a diamond (M:N)	Resolved into an associative Junction Table with FKs	Implemented with Foreign Key constraints & composite B+ Tree indexes
Data Types & Indexes	Omitted (or generic names only)	Generic logical types (Integer, String, Date)	Exact engine types (BIGINT, VARCHAR(80), TIMESTAMPTZ) + Indexes

3.4.2 Worked Example 3.2: End-to-End Walkthrough (Conceptual -> Logical -> Physical)

WORKED EXAMPLE 3.2 | Designing University Course Enrollment Across All 3 Modeling Stages

Requirement: "Each Department offers multiple Courses. Students enroll in Courses during a Semester and receive a Grade."

Stage 1 — Conceptual Model (ER Diagram):
• Entities: DEPARTMENT, COURSE, STUDENT.
• Relationships: DEPARTMENT —< OFFERS (1:N) >— COURSE; STUDENT —< ENROLLS (M:N, with attributes Semester, Grade) >— COURSE.

Stage 2 — Logical Relational Model (DBMS-Neutral Relations in 3NF):
• Department(DeptID [PK], DeptName, Building)
• Course(CourseID [PK], Title, Credits, DeptID [FK -> Department])
• Student(StudentID [PK], Name, CGPA, DeptID [FK -> Department])
• Enrollment(StudentID [FK -> Student], CourseID [FK -> Course], Semester, Grade, PRIMARY KEY(StudentID, CourseID, Semester))

Stage 3 — Physical Model (PostgreSQL-Specific DDL with Types, Constraints & Indexes):
• Specifies VARCHAR(10), NUMERIC(3,2) CHECK (CGPA BETWEEN 0 AND 10), ON DELETE RESTRICT, and B+ Tree index CREATE INDEX idx_enrollment_course ON Enrollment(CourseID, Semester);

3.5 Choosing the Right Data Model in System Design Interviews

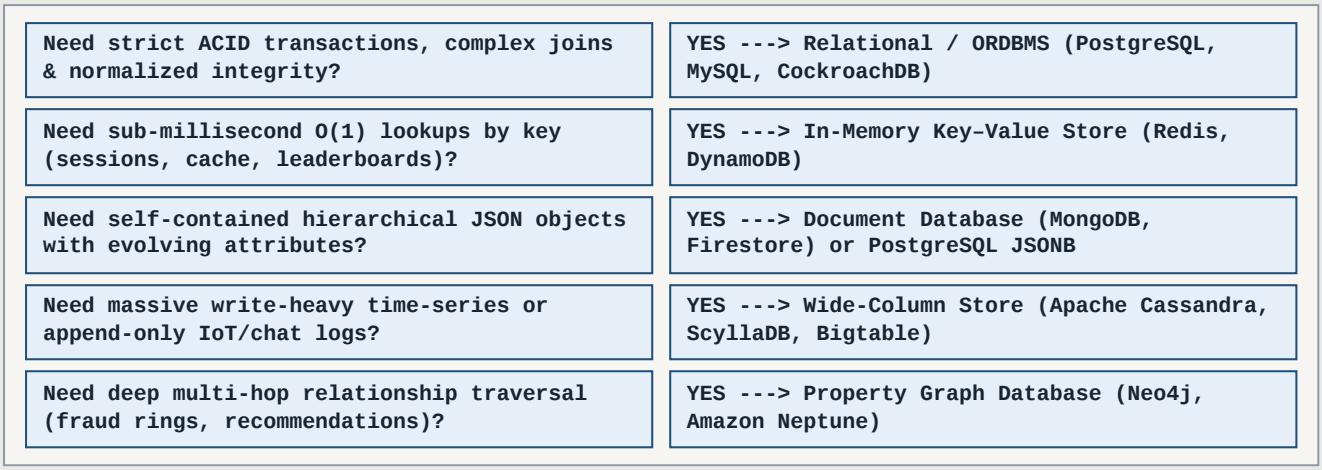


Figure 3.4: Architecture Decision Tree for Choosing the Right Data Model in System Design Interviews.

3.5.1 Worked Example 3.3: Hybrid Object-Relational Modeling in PostgreSQL

Listing 3.1: PostgreSQL Object-Relational DDL Using Composite Types, Arrays, and JSONB

```
-- 1. User-Defined Composite Type (ORDBMS Feature)
CREATE TYPE full_address AS (
    street  VARCHAR(100),
    city    VARCHAR(50),
    pincode CHAR(6)
);

-- 2. Hybrid Relational Table combining strict columns, Composite Type, Array, and JSONB
CREATE TABLE Student_Hybrid (
    StudentID    INT PRIMARY KEY,
    Name         VARCHAR(80) NOT NULL,
    HomeAddress  full_address,               -- Composite UDT
    SkillTags    TEXT[],                     -- Native Array type
    ExtraProfile JSONB                       -- Semi-structured JSONB document
);

-- 3. Creating a GIN Index on JSONB and querying nested fields alongside relational columns
CREATE INDEX idx_student_profile_gin ON Student_Hybrid USING GIN (ExtraProfile);

SELECT Name, (HomeAddress).city AS City, ExtraProfile->>'github' AS GitHub
FROM Student_Hybrid
WHERE 'PostgreSQL' = ANY(SkillTags)
  AND ExtraProfile @> '{"hackathonWinner": true}';
```

3.5.2 Worked Example 3.4: Polyglot Persistence for a Multi-Service Campus Super-App

Modern large-scale enterprises rarely force every microservice into a single data model. Instead, they practice Polyglot Persistence—choosing the best data model for each bounded service:

Polyglot Persistence Architecture for the UniVerse Campus Super-App

Microservice / Workload	Chosen Data Model & Engine	Engineering Justification
1. Tuition Fee Ledger & Exam Grades	Relational (PostgreSQL)	Requires strict multi-row ACID transactions, foreign keys, and auditability.
2. Live Login Sessions & OTP Rate Limiter	Key–Value (Redis)	Requires < 1 ms O(1) key lookups and automatic TTL expiration (EXPIRE 300s).
3. Course E-Learning Content & Dynamic Quizzes	Document (MongoDB / JSONB)	Each module has deeply nested, polymorphic blocks (videos, code labs, rubrics).
4. Campus IoT Smart-Meter & Wi-Fi Access Logs	Wide-Column (Cassandra)	50,000 sensor writes/sec partitioned by (DeviceID, Day) and sorted by Timestamp.
5. Alumni Mentorship & Research Co-Author Network	Graph (Neo4j)	Queries 3-to-5-hop co-authorship and alumni referral paths in milliseconds.

COMMON MISTAKE | Using NoSQL When You Actually Have Highly Relational Data

Storing highly interconnected financial or academic records in a Document database and then writing 6-stage application-side $lookup joins loses ACID referential integrity and performs worse than PostgreSQL. Default to Relational/ORDBMS unless a specific access pattern demands a specialized NoSQL model!

3.6 Chapter 3 Summary & One-Page Rapid-Revision Cheat Sheet

Chapter 3 Rapid-Revision Cheat Sheet

Topic / Model	Core Exam Rule / Key Distinction
3 Parts of a Data Model	1. Structural Part (data organization) + 2. Manipulative Part (operators) + 3. Integrity Constraints (rules).
3 Categories of Models	Conceptual (ER/EER) -> Representational/Logical (Relational/Network/Document) -> Physical (Slotted pages/B+ Trees).
Hierarchical Model (IMS)	Strict Tree (1:N). Child has at most 1 parent. M:N causes data duplication. Procedural navigation.
Network Model (CODASYL)	Directed Graph (Owner–Member sets). Handles M:N via intersection records, but uses complex circular pointer chains.
Relational Model (Codd)	2D tables (Relations). Links tables via value-based Foreign Keys (zero physical pointers exposed to users) + declarative SQL.
ORDBMS (PostgreSQL)	Combines Relational tables + ACID with UDTs, Arrays, JSONB, and spatial indexes (GIN/GiST).
4 NoSQL Families	Document (MongoDB: JSON), Key–Value (Redis: O(1) hash), Wide-Column (Cassandra: LSM time-series), Graph (Neo4j: nodes/edges).
Conceptual vs. Logical vs. Physical	Conceptual = Entities/Diamonds (DBMS-agnostic); Logical = Normalized Tables/PK/FK; Physical = SQL DDL, data types, indexes.

EXAM CHECKPOINT | Golden Rule for M:N Relationships Across Models

• In a Conceptual ER Diagram: Drawn directly as a diamond with M:N cardinality.
• In a Logical/Physical Relational Schema: ALWAYS requires a 3rd associative Junction Table containing the Foreign Keys of both participating tables!

Student Revision & Lecture Notes — Chapter 3

Use this page to summarize the evolution from Hierarchical -> Network -> Relational -> NoSQL and practice converting a Conceptual M:N relationship into a Logical Relational schema.

Active Recall Revision Prompts — Chapter 3

Revision Question	Key Points to Write in Exam
Why did Relational replace Hierarchical & Network?	Eliminated physical disk pointers & procedural navigation; introduced value-based Foreign Keys, declarative SQL, and physical data independence.
How does Network model represent M:N without duplication?	Uses a third intersection record type that is a member of two CODASYL Owner–Member sets (owned by both parent record types).
When should you use PostgreSQL JSONB vs. pure MongoDB?	PostgreSQL JSONB gives flexible document attributes + GIN indexing inside the same ACID database as your relational tables, avoiding distributed sync bugs.
What are the 3 outputs of Conceptual, Logical, and Physical modeling?	1. Conceptual -> ER Diagram 2. Logical -> 3NF/BCNF Relational Schema (PK/FK) 3. Physical -> Target SQL DDL + Indexes + Partitions

3.7 Chapter 3 Exam, GATE & Placement Question Bank

Part A: 10 Multiple-Choice Questions (MCQs)

1. Q1. Which of the following is NOT one of the three essential components of a formal Data Model?
   (A) Structural Part (B) Manipulative Part (C) Integrity Constraints Part (D) Network Socket Protocol Part
2. Q2. The Entity-Relationship (ER) model belongs to which category of data models?
   (A) Low-Level / Physical Data Model (B) High-Level / Conceptual Data Model (C) CODASYL Pointer Model (D) Wide-Column Physical Model
3. Q3. What is the primary structural limitation of the Hierarchical Data Model (IBM IMS)?
   (A) It cannot represent 1:N relationships (B) Every child node can have at most one parent node, making M:N relationships redundant or awkward (C) It does not support disk storage (D) It requires SQL joins
4. Q4. How does the Relational Data Model establish relationships between two tables?
   (A) Physical circular linked lists of disk addresses (B) Parent–child tree segments (C) Matching attribute values via Primary Keys and Foreign Keys (D) Index-free adjacency pointers
5. Q5. In the CODASYL Network Data Model, the basic construct used to represent a 1:N relationship between an owner record type and a member record type is called a:
   (A) Relation (B) CODASYL Set (C) BSON Document (D) Foreign Key Constraint

Part A: Multiple-Choice Questions (Continued — Q6 to Q10)

- 6. Q6. Which NoSQL database family is best suited for storing write-heavy IoT sensor telemetry partitioned by DeviceID and sorted by timestamp?
  (A) Graph Database (Neo4j) (B) Wide-Column Store (Apache Cassandra) (C) Hierarchical Mainframe (IMS) (D) In-Memory Key-Value (Memcached)
- 7. Q7. At which stage of data modeling is a Many-to-Many (M:N) relationship first decomposed into an associative junction table with Foreign Keys?
  (A) Conceptual Data Modeling (B) Logical Data Modeling (C) Physical Disk Partitioning (D) Network Socket Routing
- 8. Q8. PostgreSQL supports relational tables with ACID transactions while also allowing User-Defined Composite Types, Arrays, and indexed JSONB columns. PostgreSQL is therefore classified as an:
  (A) Hierarchical DBMS (B) Object-Relational DBMS (ORDBMS) (C) Pure Key-Value Store (D) CODASYL Network DBMS
- 9. Q9. Why do Property Graph databases (like Neo4j) outperform relational databases on 5-hop "friend-of-a-friend" traversal queries?
  (A) Graph databases do not use RAM (B) Index-Free Adjacency allows direct O(1) pointer hops between connected nodes without global B+ Tree join lookups (C) Graph databases store everything in a single CSV file (D) Relational databases cannot perform self-joins
- 10. Q10. Exact column data types (such as VARCHAR(80), NUMERIC(10,2)), B+ Tree index definitions, and table partitioning clauses belong to the:
  (A) Conceptual Data Model (B) Business Stakeholder Brief (C) Physical Data Model (D) External User View

Part B: 5 Short-Answer Questions (2–3 Marks Semester Exam Style)

1. B1. Define a Data Model and briefly explain its three core components.
2. B2. Why did the Relational Model replace the Hierarchical and Network models as the dominant enterprise database standard?
3. B3. Compare how the Hierarchical Model and the Network Model handle a Many-to-Many (M:N) relationship.
4. B4. Differentiate between a Conceptual Data Model, a Logical Data Model, and a Physical Data Model.
5. B5. When should a system architect embed related data inside a single Document (MongoDB/JSONB) vs. normalizing it into separate tables with Foreign Keys?

Part C: 3 Long-Answer / Essay Questions (5–10 Marks Semester Exam Style)

1. C1. Trace the evolution of database data models from the 1960s to the present day (Hierarchical, Network, Relational, Object-Relational, and the 4 NoSQL families). Include diagrams showing how a Student–Course relationship is represented in each.
2. C2. Explain the three stages of database design (Conceptual, Logical, and Physical Data Modeling). Using a Hospital Management System (Doctors, Patients, Prescriptions), show the exact artifacts produced at each stage.
3. C3. What is Polyglot Persistence? Design the multi-database architecture for an online food-delivery platform (like Swiggy/Zomato) covering order billing, live driver GPS tracking, restaurant menus, and coupon session caching.

Part D: 3 Quantitative & Practical Modeling Problems

1. D1. (Redundancy Calculation in Hierarchical vs. Relational Model): Suppose 10,000 Students each enroll in an average of 6 Courses. A Student profile occupies 200 bytes, a Course record occupies 120 bytes (across 500 courses), and an Enrollment link (StudentID + CourseID + Grade) occupies 20 bytes. Calculate the total storage required in: (a) a Hierarchical tree where Course is the parent and full Student records are duplicated under each Course they take; vs. (b) a Normalized Relational schema.
2. D2. (Graph Index-Free Adjacency vs. Relational B+ Tree Self-Joins): A social network has N = 16,000,000 users, and each user has an average of d = 50 friends. Finding 3-hop connections requires examining d^3 = 125,000 paths. Compare the number of B+ Tree index node comparisons in a Relational join (where each B+ Tree lookup on 16M rows requires log2(16,000,000) = 24 comparisons) against Index-Free Adjacency (1 direct pointer hop per edge).
3. D3. (Logical-to-Physical Translation): Convert the conceptual requirement "Many Authors write Many Books, and each contribution records AuthorOrder and RoyaltyPct" into (a) a Logical Relational Schema and (b) Physical PostgreSQL DDL.

Part E: 5 GATE CS Style Questions (MCQ, MSQ, and NAT)

1. E1. [GATE MCQ — 1 Mark] Which of the following data models represents all relationships exclusively through value-based matching of attributes rather than physical or logical pointers?
   (A) Hierarchical Model (B) CODASYL Network Model (C) Relational Model (D) Property Graph Model
2. E2. [GATE MSQ — 2 Marks] Which of the following statements regarding Conceptual, Logical, and Physical Data Models are CORRECT?
   (A) A Conceptual ER model can represent a Many-to-Many (M:N) relationship directly without an intermediate junction entity.
   (B) A Logical Relational model resolves every M:N relationship into a separate relation whose primary key includes foreign keys referencing the participating relations.
   (C) Adding a B+ Tree index changes the Conceptual Data Model.
   (D) The Physical Data Model specifies DBMS-specific column types, storage parameters, and indexing strategies.
3. E3. [GATE NAT — 2 Marks] In a Hierarchical database where Courses are parent segments and Student profiles (250 bytes each) are stored as child segments under every Course in which a student enrolls: if 4,000 students each enroll in 5 courses, the total storage consumed by Student child segments (in Megabytes, where 1 MB = 10^6 bytes) is _____.
4. E4. [GATE NAT — 2 Marks] If the same 4,000 students (250 bytes each) and their 5 enrollments per student are stored in a Relational database using a Student table (250 B/row) and an Enrollment table (15 B/row), the total storage consumed by Student + Enrollment rows (in Megabytes, 1 MB = 10^6 bytes) is _____.
5. E5. [GATE MCQ — 2 Marks] Match the database workload with the most appropriate primary data model:
   P. Double-entry banking ledger with ACID transfers 1. Key–Value Store (Redis)
   Q. Ephemeral shopping cart & sub-ms user session token 2. Relational DBMS (PostgreSQL)
   R. 6-hop money-laundering ring detection across accounts 3. Wide-Column Store (Cassandra)
   S. 100,000 writes/sec of smart-grid voltage readings 4. Property Graph DBMS (Neo4j)
   (A) P-2, Q-1, R-4, S-3 (B) P-1, Q-2, R-4, S-3 (C) P-2, Q-4, R-1, S-3 (D) P-3, Q-1, R-4, S-2

Part F: 5 Viva / Technical Placement Interview Questions

1. F1. "Why did E.F. Codd’s Relational Model win over the Network Model even though early 1970s relational prototypes were slower than CODASYL pointer databases?"
2. F2. "In MongoDB (Document Model), when should you EMBED child objects inside a parent document vs. REFERENCE them by ObjectId?"
3. F3. "PostgreSQL has JSONB columns and GIN indexes. Does that mean we never need MongoDB?"
4. F4. "What is the difference between Schema-on-Write (Relational) and Schema-on-Read (Document/Data Lake)?"
5. F5. "Explain how a Wide-Column database like Cassandra achieves massive write throughput compared to a B+ Tree relational table."

Part G: 2 Hands-On Modeling & SQL Lab Exercises

1. Lab 3.1 (Relational vs. JSONB Hybrid Lab in PostgreSQL): Create a table Course_Catalog that stores core relational fields (CourseID, Title, Credits, DeptID) alongside a JSONB column Metadata holding prerequisites, lab software, and grading weights. Write a SQL query that joins Course_Catalog with Department and filters on a nested JSONB key.
2. Lab 3.2 (Three-Stage Schema Design Exercise): Take a University Hostel Allotment scenario (Hostel, Room, Student, Warden) and write out (1) the Conceptual Entities & Cardinalities, (2) the Logical Relational Schema with PK/FKs, and (3) the Physical SQL DDL.

Part H: 3 "Spot the Mistake" Diagnostic Challenges

1. Mistake 3.1: A student draws B+ Tree index names and VARCHAR(255) data types inside a Conceptual ER diagram.
2. Mistake 3.2: A backend developer stores a user’s 500,000 IoT temperature readings as an unbounded embedded array inside a single MongoDB User document.
3. Mistake 3.3: An architect claims: "Foreign Keys in PostgreSQL store the physical disk sector address of the referenced parent row."

3.8 Chapter 3 Complete Answer Key & Worked Solutions

Solutions to Part A: Multiple-Choice Questions (Q1 to Q10)

Answer Key & Detailed Explanations for Part A MCQs

Q#	Ans	Detailed Step-by-Step Explanation
Q1	(D)	Every formal Data Model consists of 3 parts: Structural, Manipulative, and Integrity Constraints. Network sockets are transport layer protocols.
Q2	(B)	The ER/EER model is a High-Level Conceptual Data Model focused on business entities and relationships.
Q3	(B)	In a strict tree (Hierarchical Model), every child has at most 1 parent, so M:N relationships cause severe data duplication.
Q4	(C)	The Relational Model links tuples purely by value-based Foreign Keys matching Primary Keys—never by physical disk pointers.
Q5	(B)	A CODASYL Set links one Owner record occurrence to a ring of Member record occurrences in the Network Model.
Q6	(B)	Wide-Column stores (Cassandra/Bigtable) use LSM-Trees and partition keys + clustering timestamps for high-velocity time-series writes.
Q7	(B)	Conceptual ER diagrams show M:N directly; Logical Data Modeling decomposes M:N into an associative junction table with Foreign Keys.
Q8	(B)	PostgreSQL is the flagship open-source Object-Relational DBMS (ORDBMS).
Q9	(B)	Index-Free Adjacency stores direct edge references on each vertex, avoiding O(log N) global index lookups at each hop.
Q10	(C)	Exact engine data types (VARCHAR(80)), B+ Tree indexes, and partitioning belong to the Physical Data Model.

Solutions to Part B: Short-Answer Questions (B1 to B5)

- Answer B1 (Data Model & 3 Components): A Data Model is a formal abstraction for describing data. Its three components are: (1) Structural Part (how data is organized, e.g., tables or trees), (2) Manipulative Part (allowed query/update operations), and (3) Integrity Constraints (rules ensuring valid states, such as PK/FK rules).
- Answer B2 (Why Relational Won): Hierarchical and Network databases tightly coupled application code to physical pointer chains—if disk structures changed, every program broke. The Relational Model introduced value-based Foreign Keys, mathematical set operations (Relational Algebra), declarative SQL, and complete Physical Data Independence.
- Answer B3 (M:N in Hierarchical vs. Network): The Hierarchical Model cannot give a child two parents in a single tree, forcing record duplication under multiple parents (or virtual pointer segments). The Network Model introduces an intersection record type that participates as a member in two Owner–Member sets, avoiding data duplication at the cost of complex circular pointer rings.
- Answer B4 (Conceptual vs. Logical vs. Physical): Conceptual models business entities and relationships independent of any DBMS (ER diagram). Logical translates entities into normalized tables, columns, Primary Keys, and Foreign Keys (3NF/BCNF). Physical adds target-DBMS SQL DDL, exact column types, B+ Tree indexes, and partitioning.
- Answer B5 (Embed vs. Reference in Document Model): Embed child data inside the parent document when the relationship is 1:1 or bounded 1:Few (e.g., a student's 2 addresses or 8 semester courses) and both are always fetched together. Reference (normalize into separate collections/tables) when the child array grows unboundedly (1:Millions) or when the child entity is shared and updated independently (M:N).

Solutions to Part C: Long-Answer / Essay Questions (C1 to C3)

Model Solution C1 | Evolution of Data Models (1960s -> Modern NoSQL)

1. Era 1 — Navigational Pointer Models (1960s–1970s): Hierarchical (IBM IMS) modeled 1:N trees; Network (CODASYL) modeled directed graphs via circular pointer rings. Fast for fixed queries, but zero physical data independence.
2. Era 2 — Relational & Object-Relational Models (1970–1990s): E.F. Codd's Relational Model replaced pointers with 2D mathematical relations, Foreign Keys, and declarative SQL. ORDBMS (PostgreSQL, Oracle) added UDTs, Arrays, JSONB, and spatial indexing.
3. Era 3 — Specialized NoSQL Models (2000s–Present): Document (MongoDB), Key–Value (Redis), Wide-Column (Cassandra), and Graph (Neo4j) complement RDBMSs in polyglot architectures.

Model Solution C2 & C3 | Hospital 3-Stage Modeling & Food-Delivery Polyglot Architecture

Hospital 3-Stage Summary (C2):
• Conceptual: Entities DOCTOR, PATIENT, MEDICINE connected by M:N relationship PRESCRIPTION (with Date, Dosage).
• Logical: Doctor(DocID PK, Name, Spec), Patient(PatID PK, Name, DOB), Prescription(PrescID PK, DocID FK, PatID FK, IssuedDate), Prescription_Item(PrescID FK, MedID FK, Dosage).
• Physical: PostgreSQL DDL with TIMESTAMPTZ, Foreign Key ON DELETE RESTRICT, and B+ Tree index on Prescription(PatID, IssuedDate).

Food-Delivery Polyglot Summary (C3):
• Order Billing & Wallet Payments -> PostgreSQL (ACID transactions).
• Live Delivery Partner GPS Ping & Surge Cache -> Redis (Key–Value + GeoSpatial).
• Restaurant Menus with Custom Add-on Modifiers -> MongoDB / PostgreSQL JSONB (Document).
• Historical Delivery Telemetry & Chat Logs -> Cassandra (Wide-Column).

Solutions to Part D: Quantitative & Practical Modeling Problems (D1 to D3)

Worked Solution D1 | Storage Redundancy: Hierarchical vs. Relational

Given: 10,000 Students (200 B each), 500 Courses (120 B each), average 6 courses per student (60,000 enrollments; Grade/Link = 20 B).
(a) Hierarchical Tree (Course as Parent, full 200 B Student + 20 B Grade duplicated under each Course):
• Course segments = 500 × 120 B = 60,000 B (0.06 MB).
• Duplicated Student child segments = 60,000 enrollments × (200 + 20) B = 13,200,000 B (13.20 MB).
• Total Hierarchical Storage = 13,260,000 bytes (13.26 MB).

(b) Normalized Relational Schema (Student stored once, Course stored once, Enrollment stores 20 B link):
• Student table = 10,000 × 200 B = 2,000,000 B (2.00 MB).
• Course table = 500 × 120 B = 60,000 B (0.06 MB).
• Enrollment junction table = 60,000 × 20 B = 1,200,000 B (1.20 MB).
• Total Relational Storage = 3,260,000 bytes (3.26 MB) -> Saves 10.0 MB (75.4% storage reduction) and eliminates update anomalies!

Worked Solution D2 | Graph Index-Free Adjacency vs. Relational B+ Tree Self-Joins

Given: N = 16,000,000 users (log2 N = 24), degree d = 50, 3-hop paths = d^3 = 125,000.
• Property Graph (Index-Free Adjacency): Traverses 125,000 direct edge pointers = 1.25 × 10^5 pointer ops.
• Relational B+ Tree Join: Each lookup across 16M rows performs ~24 key comparisons -> 125,000 × 24 = 3,000,000 comparisons (24× more index traversal work!).

Solutions to Part D (D3) and Part E: GATE CS Questions (E1 to E5)

Solution D3: Logical Relational Schema & Physical PostgreSQL DDL for Author–Book (M:N)

```
-- Logical Schema:
-- Author(AuthorID [PK], FullName, ORCID)
-- Book(ISBN [PK], Title, PubYear)
-- Book_Authorship(ISBN [FK], AuthorID [FK], AuthorOrder, RoyaltyPct, PK(ISBN, AuthorID))

CREATE TABLE Author (
    AuthorID SERIAL PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    ORCID    CHAR(19) UNIQUE
);
CREATE TABLE Book (
    ISBN    CHAR(13) PRIMARY KEY,
    Title   VARCHAR(200) NOT NULL,
    PubYear INT CHECK (PubYear >= 1900)
);
CREATE TABLE Book_Authorship (
    ISBN        CHAR(13) REFERENCES Book(ISBN) ON DELETE CASCADE,
    AuthorID    INT REFERENCES Author(AuthorID) ON DELETE RESTRICT,
    AuthorOrder SMALLINT NOT NULL CHECK (AuthorOrder >= 1),
    RoyaltyPct  NUMERIC(5,2) NOT NULL CHECK (RoyaltyPct BETWEEN 0 AND 100),
    PRIMARY KEY (ISBN, AuthorID)
);
```

Solutions & Derivations for Part E: GATE CS Questions (E1 to E5)

GATE Q#	Correct Answer	Step-by-Step Derivation
E1 (MCQ)	(C) Relational Model	Only the Relational Model represents all relationships purely by matching attribute values (PK = FK) without pointers.
E2 (MSQ)	(A), (B), (D)	(C) is false: adding a B+ Tree index modifies the Physical/Internal model, not the Conceptual ER model.
E3 (NAT)	5 (or 5.0)	4,000 students × 5 courses × 250 bytes = 5,000,000 bytes = 5.0 MB.
E4 (NAT)	1.3	Student: 4,000 × 250 B = 1,000,000 B (1.0 MB) + Enrollment: 20,000 × 15 B = 300,000 B (0.3 MB) = 1.3 MB total.
E5 (MCQ)	(A) P-2, Q-1, R-4, S-3	Banking Ledger -> Relational (2); Session Token -> Key–Value (1); 6-hop Fraud -> Graph (4); IoT Time-Series -> Wide-Column (3).

Solutions to Part F: Viva & Technical Placement Interview Questions (F1 to F5)

- Answer F1 (Why Relational Beat CODASYL Despite Early Speed): In the 1970s, hardware became cheaper while software engineering labor became the dominant cost. CODASYL databases required weeks of manual C/COBOL pointer rewriting whenever business requirements changed. As Cost-Based Query Optimizers (IBM System R) and B+ Tree indexes matured, Relational databases matched navigational speed while giving 10× higher developer productivity and Physical Data Independence.
- Answer F2 (MongoDB Embed vs. Reference Rule of Thumb): Embed when the child data is owned exclusively by the parent (1:1 or 1:Few, e.g., < 50 sub-items), read together 95% of the time, and rarely updated independently. Reference by ID when the relationship is 1:Many (unbounded growth, e.g., comments or sensor logs) or M:N where independent updates would otherwise require rewriting thousands of documents.
- Answer F3 (PostgreSQL JSONB vs. Dedicated MongoDB): For 90% of startups and enterprise apps, PostgreSQL + JSONB is superior because you get ACID relational joins AND indexed semi-structured documents in one system. Dedicated MongoDB shines primarily when your entire application is document-centric and requires built-in horizontal auto-sharding across dozens of write nodes.
- Answer F4 (Schema-on-Write vs. Schema-on-Read): Schema-on-Write (RDBMS) validates every tuple against a strict catalog schema before writing bytes to disk—guaranteeing clean data quality. Schema-on-Read (Document DBs / Data Lakes) accepts arbitrary JSON/files on ingest and interprets structure when the application reads it—faster initial prototyping, but requires defensive validation in application code.
- Answer F5 (Why Cassandra Writes So Fast): Whereas a B+ Tree updates existing disk pages in place (random disk I/O), Cassandra uses an LSM-Tree (Log-Structured Merge-Tree): every write is appended sequentially to an on-disk CommitLog and inserted into an in-memory Memtable, later flushed sequentially as immutable SSTables.

Solutions to Part G (Labs 3.1–3.2) & Part H ("Spot the Mistake" 3.1–3.3)

Solution Lab 3.1: Hybrid Relational + JSONB Catalog Query in PostgreSQL

```
CREATE TABLE Course_Catalog (
    CourseID VARCHAR(10) PRIMARY KEY,
    Title    VARCHAR(100) NOT NULL,
    Credits  INT NOT NULL,
    DeptID   INT REFERENCES Department(DeptID),
    Metadata JSONB NOT NULL DEFAULT '{}'::jsonb
);
CREATE INDEX idx_course_meta_gin ON Course_Catalog USING GIN (Metadata);

SELECT c.CourseID, c.Title, d.DeptName, c.Metadata->>'labSoftware' AS LabTool
FROM Course_Catalog c
JOIN Department d ON c.DeptID = d.DeptID
WHERE c.Metadata @> '{"hasLab": true}';
```

Diagnostic Solutions 3.1, 3.2, and 3.3

• Mistake 3.1 (Physical Details in Conceptual ER): B+ Tree index names and VARCHAR(255) belong to the Physical Data Model, never the Conceptual ER model.
• Mistake 3.2 (Unbounded Array Anti-Pattern in MongoDB): MongoDB enforces a hard 16 MB limit per BSON document, and appending 500,000 items forces constant document reallocation on disk. Unbounded time-series must be stored as separate documents (or in a Wide-Column/Time-Series DB).
• Mistake 3.3 (Foreign Keys Are Values, Not Pointers): A Foreign Key in a Relational DBMS stores the logical Primary Key value (e.g., DeptID = 10), never a physical disk sector address!

END OF CHAPTER 3 | Up Next: Chapter 4 — Entity-Relationship (ER) and Enhanced ER (EER) Modeling

In Chapter 4, we master Chen and Crow's Foot ER diagrams, weak entities, EER specialization/generalization/aggregation, the complete 9-step ER-to-Relational mapping algorithm, and the GATE CS formula for finding the minimum number of tables!